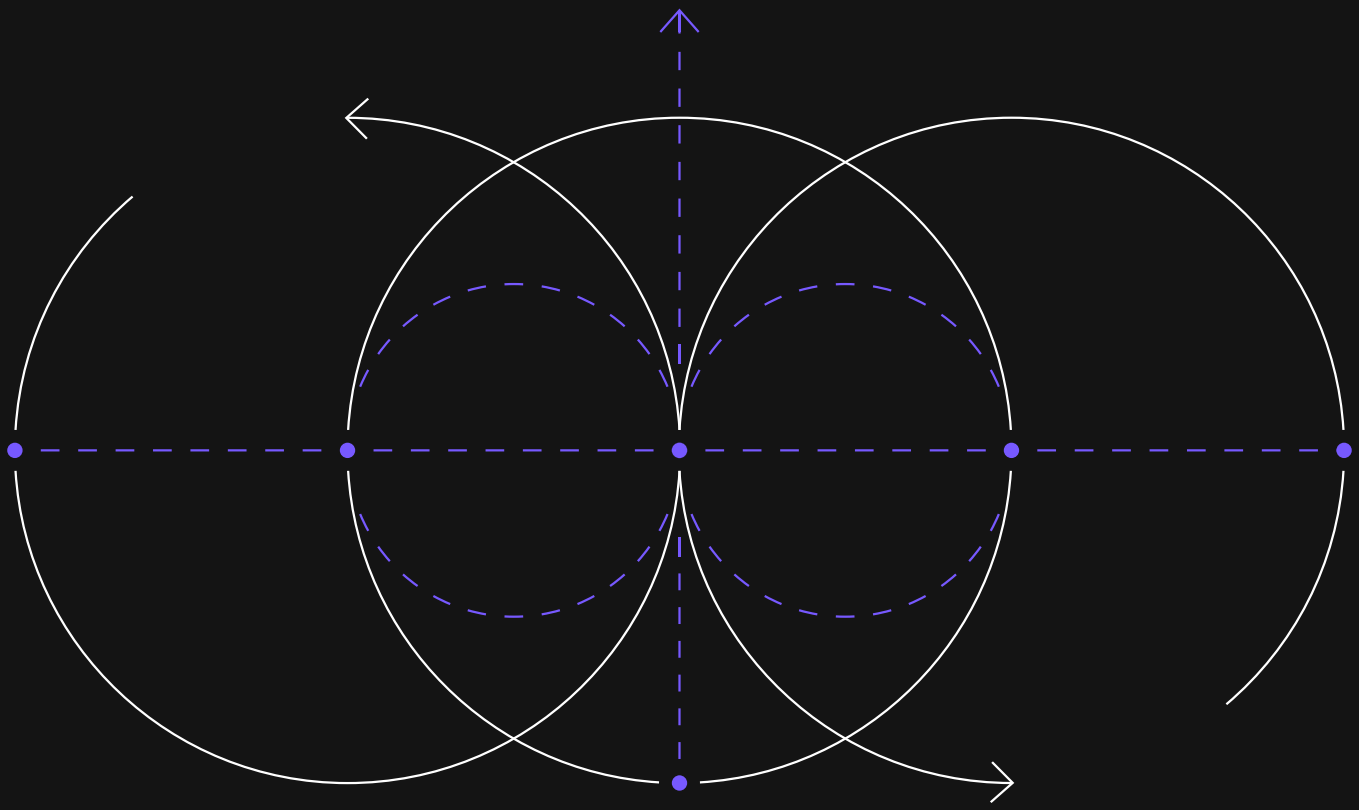
Машинное обучение

Лонгрид

Глубокое обучение

09

Эффективное дообучение и инференс LLM



Содержание

1. Постановка задачи и обозначения 3
2. Полное дообучение как базовый метод 4
3. Низкоранговая адаптация LoRA 5
3.1. Низкоранговое обновление весов 5
3.2. Инициализация и обучение 5
3.3. Выбор матриц и ранга 6
3.4. Слияние весов 6
3.5. Память и вычислительная стоимость 7
4. QLoRA 7
4.1. Хранение и вычисление весов 7
4.2. NF4 и двойное квантование 8
4.3. Память обучения и ограничения 8
5. Авторегрессионный инференс 9
5.1. Обработка входа и пошаговая генерация 9
5.2. KV-кеш 10
6. FlashAttention 10
6.1. Материализация матрицы внимания 10
6.2. Потоковое вычисление softmax 11
6.3. Блочное вычисление внимания 12
6.4. Память, обмен данными и ограничения 12
7. Сравнение вычислительных затрат 13
8. Оценка качества и эффективности 14
9. Практические ограничения 15
9.1. Ограничение ранга 15
9.2. Квантизация базовой модели 15
9.3. Активации и длина последовательности 15
9.4. Режим инференса 15
10. Основные выводы 15
A. Пример расчёта памяти обучения 17
B. Проверка потокового softmax 17

Эффективное дообучение и инференс LLM

В первой части недели функция потерь SFT задавалась для полного обновления параметров модели. Здесь функция потерь остаётся прежней, но изменяется множество обучаемых параметров. Рассматриваются LoRA и QLoRA: первая задаёт низкоранговую поправку к весам, вторая дополнительно хранит замороженную базовую модель в квантизованном виде.

Вторая тема — вычисление decoder-only-трансформера при обработке входа и пошаговой генерации. KV-кеш кратко фиксируется как уже пройденный механизм, после чего разбирается FlashAttention. Он вычисляет обычное плотное внимание без материализации полной матрицы вероятностей во внешней памяти. Архитектура трансформера, SFT и варианты GQA и MLA заново не выводятся.

1. Постановка задачи и обозначения

Пусть имеются предобученная модель с параметрами θ_0 и выборка

$$\underset{\text{SFT}}{\mathsf{D}} = \{(x_i, y_i)\}_{i=1}^{n},$$

где x_i — запрос, а y_i — правильный ответ. Условная вероятность ответа обозначается через $\pi_\theta(y \mid x)$. Функция потерь SFT определена в первой части недели. В этой лекции она записывается кратко как

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y)\sim \mathsf{D}_{\text{SFT}}} \sum_{t=1}^{|y|} \log \pi_\theta(y_t \mid x, y_{<t}).$$

Токены запроса входят в условие, а сумма считается по токенам ответа. LoRA не меняет эту целевую функцию. Меняется параметризация θ и число переменных, по которым берётся градиент.

Для учёта ресурсов введём обозначения:

- P — число параметров базовой модели;
- B — размер батча, L — число слоёв;
- T_x — длина запроса, T_y — число генерируемых токенов;
- n_h — число голов внимания, d_h — размер головы;
- s — число байтов на один элемент KV-кеша;
- M_{act} — память промежуточных активаций при обучении.

Буква N в материалах недели 9 обозначает число параметров в законах масштабирования, поэтому для длины последовательности она не используется.

Разбираются две независимые задачи эффективности. Первая возникает при дообучении: требуется уменьшить число обучаемых и сохраняемых параметров при контролируемом качестве. Вторая возникает при вычислении внимания: требуется сократить промежуточную память и обмен данными, не меняя результат плотного внимания.

Механизм	Что изменяется	Что остаётся прежним
LoRA	Параметризация обновления весов	Базовые веса, данные и функция потерь
QLoRA	Формат хранения базовых весов и параметры адаптера	Базовые веса остаются замороженными; вычисления выполняются после деквантизации
KV-кеш	Сохраняются ключи и значения префикса	Внимание нового токена обращается ко всему доступному контексту
FlashAttention	Порядок вычисления и размещение промежуточных данных	Математическое определение плотного внимания

Таблица 1. Механизмы действуют на разные составляющие вычисления. Их экономию нельзя описать одной долей от общей памяти или времени

2. Полное дообучение как базовый метод

При полном дообучении все параметры инициализируются значениями θ_0 и обновляются:

$$\theta^* = \arg\min_{\theta} \mathcal{L}_{\text{SFT}}(\theta).$$

Для каждого параметра обычно требуется хранить сам вес, его градиент и состояния оптимизатора. Обозначим число байтов на вес, градиент и два состояния Adam через s_w, s_g, s_{m_1} и s_{m_2}. Символическая оценка памяти имеет вид

$$M_{\text{full}} \approx P(s_w + s_g + s_{m_1} + s_{m_2}) + M_{\text{act}}.$$

В конкретной реализации может дополнительно храниться полная копия весов для обновления в другом числовом формате. Поэтому по одному названию формата весов нельзя восстановить полную память обучения.

После дообучения для каждой задачи получается отдельный набор из P параметров. Если требуется хранить K специализированных моделей, размер их весов растёт как KPs_w. При использовании общей базовой модели и небольших адаптеров набор параметров для конкретной задачи может быть существенно меньше.

Полное дообучение остаётся базовой точкой сравнения. Оно не ограничивает ранг изменения весов и не требует выбора мест для адаптеров. При сравнении с LoRA необходимо фиксировать исходную модель, данные, функцию потерь, число шагов и процедуру оценки. Сравнение только числа обучаемых параметров не показывает качество и не учитывает память активаций.

3. Низкоранговая адаптация LoRA

3.1. НИЗКОРАНГОВОЕ ОБНОВЛЕНИЕ ВЕСОВ

Рассмотрим одну линейную проекцию:

$$h = W_0 x, \qquad W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}, \qquad x \in \mathbb{R}^{d_{\text{in}}}.$$

При полном дообучении матрица становится $W_0 + \Delta W$, причём ΔW имеет ту же размерность. LoRA задаёт поправку произведением двух матриц:

$$A_{\text{L}} \in \mathbb{R}^{r \times d_{\text{in}}}, \qquad B_{\text{L}} \in \mathbb{R}^{d_{\text{out}} \times r}, \qquad \Delta W = \frac{\alpha}{r} B_{\text{L}} A_{\text{L}},$$

где $r \ll \min(d_{\text{in}}, d_{\text{out}})$. Прямой проход имеет вид

$$h = W_0 x + \frac{\alpha}{r} B_{\text{L}} (A_{\text{L}} x).$$

Матрица W_0 заморожена. Обучаются только A_{L} и B_{L}. Малоранговой является поправка ΔW, причём

$$\text{rank}(\Delta W) \le r.$$

Исходная матрица W_0 и итоговая сумма $W_0 + \Delta W$ не обязаны иметь малый ранг.

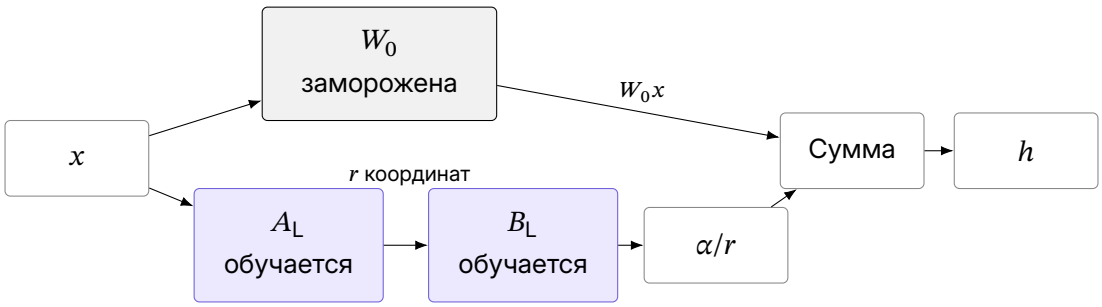


Рисунок 1. Параллельная ветвь LoRA для одной линейной проекции. Схема соответствует параметризации Hu и соавт. [1]

Для одного адаптера число обучаемых параметров равно

$$P_{\text{LoRA}} = r(d_{\text{in}} + d_{\text{out}})$$

вместо $d_{\text{in}} d_{\text{out}}$ элементов полной матрицы. Для модели с несколькими адаптированными проекциями эта величина суммируется по всем выбранным матрицам. Общая запись через d_{in} и d_{out} нужна потому, что после GQA проекции ключей и значений могут быть неквадратными.

3.2. ИНИЦИАЛИЗАЦИЯ И ОБУЧЕНИЕ

В исходной работе A_{L} инициализируется случайной гауссовской матрицей, а B_{L} — нулевой. Поэтому в начале

$$\Delta W = 0$$

и выход слоя совпадает с выходом исходной модели. Коэффициент α/r задаёт масштаб ветви LoRA. Он снижает необходимость полностью менять настройку шага обучения при смене r, но не гарантирует одинаковую динамику оптимизации для всех рангов.

Пусть $\delta = \partial \mathcal{L} / \partial h$. Тогда для одного объекта

$$\frac{\partial \mathcal{L}}{\partial B_{\mathsf{L}}} = \frac{\alpha}{r}\, \delta (A_{\mathsf{L}} x)^{\mathsf{T}}, \qquad \frac{\partial \mathcal{L}}{\partial A_{\mathsf{L}}} = \frac{\alpha}{r}\, B_{\mathsf{L}}^{\mathsf{T}} \delta x^{\mathsf{T}}.$$

Градиент по W_0 не сохраняется, но градиент по входу проходит через замороженную основную ветвь. При нулевой инициализации B_{L} градиент A_{L} на первом шаге равен нулю; после обновления B_{L} обучаются обе матрицы.

Для всей модели параметры адаптера обозначим через ϕ. Оптимизационная задача записывается как

$$\phi^* = \arg\min_{\phi} \mathcal{L}_{\mathsf{SFT}}(\theta_0, \phi), \qquad \phi = \{A_{\mathsf{L}}^{(k)}, B_{\mathsf{L}}^{(k)}\}_{k \in I},$$

где $\mathcal{I}$ — набор выбранных проекций. Параметры θ_0 фиксированы.

3.3. ВЫБОР МАТРИЦ И РАНГА

LoRA можно добавить к проекциям запросов, ключей, значений, выходной проекции внимания и линейным слоям FFN. Выбор определяет число параметров и множество допустимых обновлений. В экспериментах исходной статьи на GPT-3 175B при фиксированном бюджете адаптация проекций запросов и значений оказалась сильнее ряда исследованных альтернатив. Этот результат относится к конкретной модели, задачам и бюджету и не задаёт универсальное правило [1].

Ранг r является гиперпараметром. Малое значение сильнее ограничивает ΔW и уменьшает размер адаптера. Увеличение r расширяет множество допустимых поправок, но само по себе не гарантирует улучшения на отложенных данных. Ранг и набор проекций выбираются по валидационной выборке при фиксированном вычислительном бюджете.

3.4. СЛИЯНИЕ ВЕСОВ

После обучения одного адаптера можно вычислить

$$W_{\text{merge}} = W_0 + \frac{\alpha}{r} B_{\mathsf{L}} A_{\mathsf{L}}.$$

Дальнейший прямой проход использует одну матрицу W_{merge}, поэтому отдельная ветвь LoRA не добавляет умножений на инференсе. Если разные объекты в одном батче должны использовать разные адаптеры, слияние неудобно: адаптеры приходится применять отдельными ветвями или группировать запросы.

Для QLoRA слияние сложнее. Базовые веса сначала требуется деквантизовать, сложить с поправкой и при необходимости квантизовать снова. Поэтому отсутствие дополнительной задержки относится к конкретному режиму развёртывания, а не ко всякому использованию адаптера.

3.5. ПАМЯТЬ И ВЫЧИСЛИТЕЛЬНАЯ СТОИМОСТЬ

Обозначим через P_{L} число параметров всех адаптеров. Символическая оценка памяти LoRA равна

$$M_{\mathrm{LoRA}} \approx P s_{\mathrm{base}} + P_{\mathrm{L}}(s_a + s_g + s_{m_1} + s_{m_2}) + M_{\mathrm{act}},$$

где s_a — число байтов на обучаемый параметр адаптера. Базовые веса остаются в памяти. Для них не хранятся градиенты и состояния оптимизатора, но активации и обратный проход через замороженные слои сохраняются. Для одного входного вектора основное умножение $W_0 x$ не исчезает. Ветвь LoRA добавляет два умножения порядка

$$O(r d_{\mathrm{in}} + r d_{\mathrm{out}}).$$

Фактическое время зависит от размера батча, длины последовательности, выбранных проекций, реализации ядер и обмена данными. Доля обучаемых параметров не равна доле времени или общей памяти.

4. QLoRA

4.1. ХРАНЕНИЕ И ВЫЧИСЛЕНИЕ ВЕСОВ

QLoRA хранит замороженные базовые веса в 4-битном формате и обучает LoRA-адаптеры в более точном формате. Для одной проекции

$$\widehat{W}_0 = \mathrm{dequant}(W_0^{\mathrm{NF4}}, c), \qquad h = \widehat{W}_0 x + \frac{\alpha}{r} B_{\mathrm{L}} A_{\mathrm{L}} x,$$

где W_0^{NF4} — квантизованные значения, а c — масштабы блоков. Веса деквантизуются в вычислительный формат, обычно BF16, перед умножением. Адаптеры также хранятся и вычисляются в более точном формате. Градиенты вычисляются только для A_{L} и B_{L} [2].

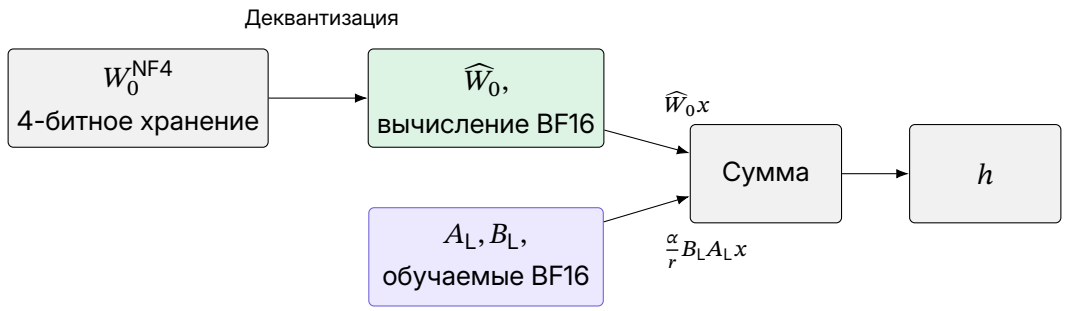


Рисунок 2. Хранение и вычисление одной проекции в QLoRA. Замороженные веса хранятся в NF4, а матричное умножение выполняется после деквантизации. Схема соответствует методу Dettmers и соавт. [2]

При нулевой поправке LoRA слой QLoRA совпадает с квантизованной аппроксимацией исходного слоя, но не обязательно с исходной BF16-матрицей:

$$\widehat{W}_0 \neq W_0.$$

Разность определяется ошибкой квантования. Поэтому QLoRA одновременно вводит ограничение низкого ранга и приближение базовых весов.

4.2. NF4 И ДВОЙНОЕ КВАНТОВАНИЕ

При блочном квантовании элементы блока $w = (w_1, \ldots, w_g)$ нормируются масштабом c, заменяются ближайшими значениями кодовой таблицы и восстанавливаются как

$$q_i = Q(w_i/c), \qquad \hat{w}_i = cq_i.$$

В NF4 уровни кодовой таблицы строятся по квантилям стандартного нормального распределения и затем нормируются в диапазон $[-1, 1]$. Такая конструкция мотивирована приблизительно нормальным распределением весов предобученной модели. Она не является безусловно оптимальной для произвольного тензора. Ноль включается в представление явно [2].

Для каждого блока требуется хранить масштаб c. Если блок содержит 64 веса, а масштаб записан 32-битным числом, накладные расходы равны

$$\frac{32}{64} = 0{,}5 \text{ бита на один вес.}$$

Двойное квантование квантизует сами масштабы первого уровня. В конфигурации статьи средние накладные расходы на масштабы уменьшаются примерно до $0{,}127$ бита на параметр. Это экономия хранения, а не отдельное изменение функции потерь.

Страничный оптимизатор в QLoRA использует объединённую память для переноса состояний оптимизатора между памятью ускорителя и оперативной памятью при пиковых нагрузках. Он помогает избежать некоторых ошибок нехватки памяти, но не является гарантированным ускорением и не относится к определению NF4.

4.3. ПАМЯТЬ ОБУЧЕНИЯ И ОГРАНИЧЕНИЯ

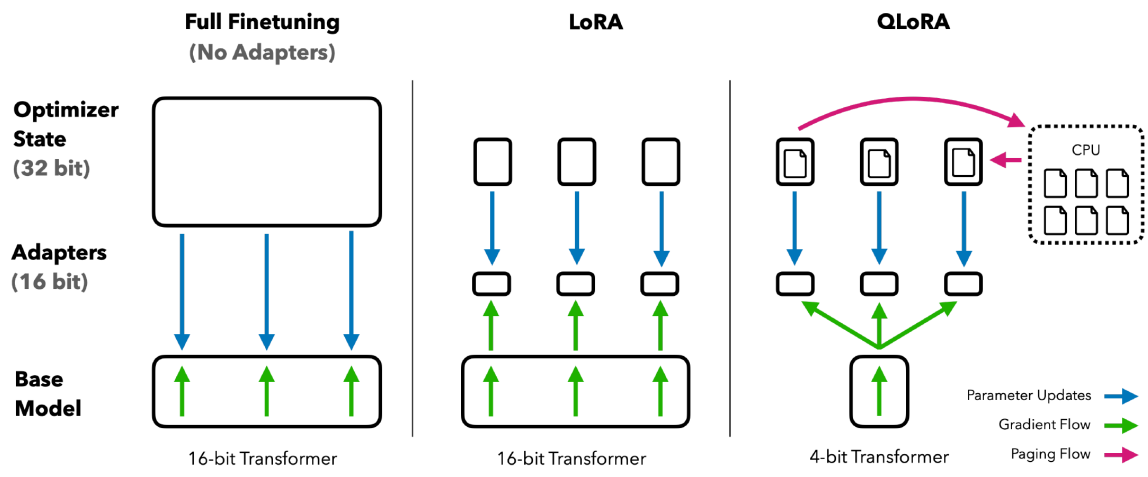


Рисунок 3. Сравнение полного дообучения, LoRA и QLoRA по хранению базовой модели, адаптеров и состояний оптимизатора. Розовые стрелки показывают перенос состояний оптимизатора при пиковых нагрузках, а не их постоянное хранение на CPU. Рисунок 1 из работы Dettmers и соавт. [2]

Если служебные данные квантования требуют в среднем δ бит на параметр, память базовых весов оценивается как

$$M_{\text{base}}^{\text{QLoRA}} \approx P\frac{4+\delta}{8} \text{ байт.}$$

К этой величине добавляются адаптеры, их градиенты и состояния оптимизатора, активации и рабочая память деквантизации. Полная BF16-копия базовой модели постоянно не хранится: нужные блоки деквантизуются при вычислении.

QLoRA уменьшает память весов, но не делает всё обучение 4-битным. Матричные операции выполняются в более точном формате, а активации могут оставаться основным потребителем памяти. Качество зависит от базовой модели, схемы квантования, набора адаптируемых проекций, ранга и данных. Совпадение с полным дообучением, показанное в экспериментах статьи, является эмпирическим результатом конкретных постановок [2].

5. Авторегрессионный инференс

5.1. ОБРАБОТКА ВХОДА И ПОШАГОВАЯ ГЕНЕРАЦИЯ

Пусть запрос содержит T_x токенов, а модель должна сгенерировать T_y токенов ответа. Сначала все позиции запроса обрабатываются параллельно с причинной маской. Этот этап называется обработкой входа. Затем токены ответа строятся последовательно. На шаге g текущая длина контекста равна

$$T_g = T_x + g - 1, \qquad g = 1, \ldots, T_y.$$

Получение ответа требует T_y последовательных решений о выборе токена, потому что токен y_g входит в контекст для y_{g+1}. Логиты первого токена уже получены при обработке входа. Поэтому после неё нужны $T_y - 1$ однотокенных проходов для вычисления токенов $y_2, \ldots, y_{T_y}$, если не выполнять лишний проход после последнего токена [3].

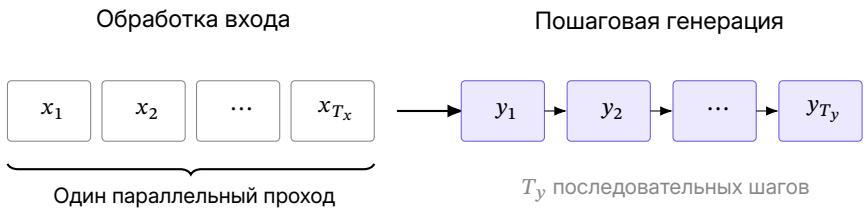


Рисунок 4. Два режима авторегрессионного инференса. Параллелизм обработки входа не устраняет последовательную зависимость между выходными токенами

5.2. KV-КЕШ

Для позиции нового токена запрос нужен один раз, а ключи и значения предыдущих позиций не меняются. KV-кеш сохраняет их после вычисления и дописывает одну новую позицию на каждом шаге. Для обычного многоголового внимания объём кеша равен

$$M_{\text{MHA}} = 2BTLn_h d_h s,$$

где T — текущая длина контекста. Множитель 2 соответствует ключам и значениям. Формула, а также GQA и MLA подробно разобраны на неделе 8.

KV-кеш устраняет повторное вычисление ключей и значений префикса, но не делает внимание нового токена постоянным по времени. Новый запрос сравнивается со всеми T сохранёнными ключами и объединяет все значения. Для одного шага плотного внимания стоимость по всем слоям имеет порядок

$$O(BLn_h T d_h).$$

Таким образом, память кеша и стоимость внимания одного шага линейно растут с текущей длиной контекста.

	Обработка входа	**Пошаговая генерация**
Форма запросов	Все T_x позиций	Одна новая позиция на последовательность
Параллелизм	По позициям входа	Между последовательностями и головами, но не по будущим токенам
Основное состояние	Активации текущего прохода	Веса модели и растущий KV-кеш
Типичная метрика	Время до первого токена	Время на токен и токены в секунду

Таблица 2. Обработка входа и генерация имеют разные формы матриц и разные ограничения производительности

6. FlashAttention

6.1. МАТЕРИАЛИЗАЦИЯ МАТРИЦЫ ВНИМАНИЯ

Для одной головы обозначим

$$Q_i, K_i, V_i \in \mathbb{R}^{T \times d_h}.$$

Плотное внимание определяется следующими формулами:

$$R_i = \frac{Q_i K_i^{\top}}{\sqrt{d_h}}, \qquad A_i = \text{softmax}_{\text{row}}(R_i), \qquad O_i = A_i V_i.$$

Причинная маска запрещает элементы выше диагонали перед softmax. Материализующая реализация записывает R_i и A_i размера $T \times T$ во внешнюю память ускорителя. Эти промежуточные данные занимают $O(T^2)$ элементов на голову. Само математическое определение внимания не требует, чтобы обе матрицы одновременно хранились во внешней памяти.

Ускоритель имеет большую внешнюю память HBM и значительно меньшую быструю память на кристалле, которую далее кратко назовём SRAM. Матричные умножения эффективно используют вычислительные блоки, а softmax, маска и поэлементные операции требуют чтения и записи промежуточных матриц. FlashAttention меняет разбиение вычисления так, чтобы блоки Q, K и V обрабатывались в SRAM, а полные R_i и A_i не записывались в HBM [4].

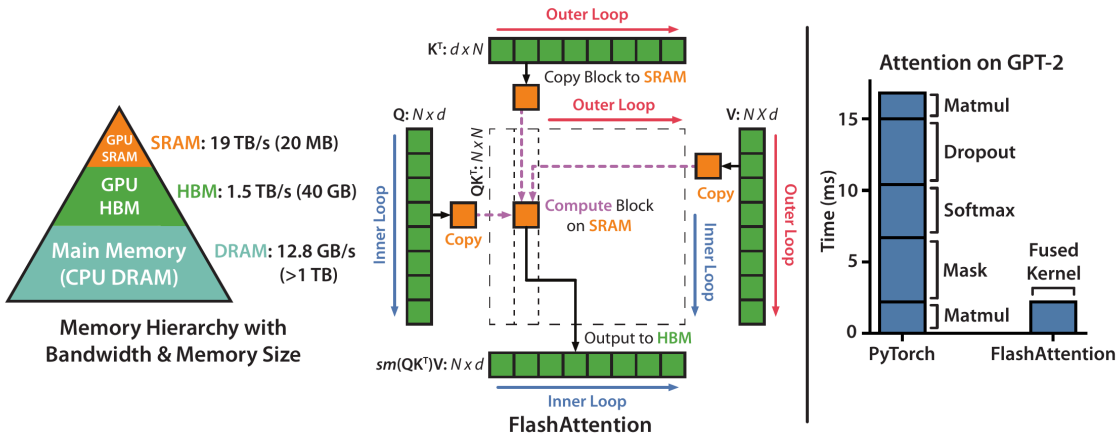


Рисунок 5. Иерархия памяти, блочное вычисление внимания и измерение времени в эксперименте статьи FlashAttention. Диаграмма справа относится к GPT-2 и конкретной программно-аппаратной конфигурации работы, поэтому приведённый коэффициент ускорения не является универсальным. Рисунок 1 из работы Dao и соавт. [4]

6.2. ПОТОКОВОЕ ВЫЧИСЛЕНИЕ SOFTMAX

Для фиксированной позиции t и головы i логит внимания равен

$$r_{t,j,i} = \frac{q_{t,i}^{\top} k_{j,i}}{\sqrt{d_h}}, \qquad j \le t.$$

Разобьём допустимые индексы на блоки $J_t^{(1)}, \ldots, J_t^{(R_t)}$. После обработки b блоков будем хранить максимум $m_{t,i}^{(b)}$, нормирующую сумму $\ell_{t,i}^{(b)}$ и ненормированный числитель $\tilde{o}_{t,i}^{(b)} \in \mathbb{R}^{d_h}$. Начальные значения равны

$$m_{t,i}^{(0)} = -\infty, \qquad \ell_{t,i}^{(0)} = 0, \qquad \tilde{o}_{t,i}^{(0)} = 0.$$

Для очередного блока вычисляются

$$m_{t,i}^{(b)} = \max\left(m_{t,i}^{(b-1)}, \max_{j \in J_t^{(b)}} r_{t,j,i}\right),$$
$$\ell_{t,i}^{(b)} = e^{m_{t,i}^{(b-1)} - m_{t,i}^{(b)}} \ell_{t,i}^{(b-1)} + \sum_{j \in J_t^{(b)}} e^{r_{t,j,i} - m_{t,i}^{(b)}},$$
$$\tilde{o}_{t,i}^{(b)} = e^{m_{t,i}^{(b-1)} - m_{t,i}^{(b)}} \tilde{o}_{t,i}^{(b-1)} + \sum_{j \in J_t^{(b)}} e^{r_{t,j,i} - m_{t,i}^{(b)}} v_{j,i}.$$

После последнего блока выход равен

$$o_{t,i} = \frac{\tilde{o}_{t,i}^{(R_t)}}{\ell_{t,i}^{(R_t)}}.$$

При увеличении текущего максимума прежняя сумма и прежний числитель домножаются на один и тот же коэффициент. Поэтому все слагаемые остаются в общем численном масштабе. Процедура не содержит обучаемых параметров и вычисляет тот же softmax, что и обычная формула. Полная проверка инварианта приведена в приложении.

6.3. БЛОЧНОЕ ВЫЧИСЛЕНИЕ ВНИМАНИЯ

FlashAttention делит строки Q_i на блоки запросов, а строки K_i и V_i — на блоки ключей и значений. Для пары блоков в SRAM вычисляются локальные логиты, локальный максимум и вклад в выход. Затем статистики каждого блока запросов обновляются по формулам потокового softmax.

Причинная маска сохраняется. Блоки, полностью лежащие выше диагонали, можно пропустить. В диагональных блоках запрещённые элементы получают значение $-\infty$ до softmax. Число допустимых пар примерно вдвое меньше, чем без маски, но асимптотика плотного внимания по длине остаётся квадратичной.

При обучении для обратного прохода не сохраняются полные R_i и A_i. Они пересчитываются блоками из Q_i, K_i, V_i и сохранённых статистик нормировки. Пересчёт добавляет операции, но сокращает обмен с HBM и может уменьшать общее время [4].

6.4. ПАМЯТЬ, ОБМЕН ДАННЫМИ И ОГРАНИЧЕНИЯ

Для одной головы число арифметических операций остаётся

$$O(T^2 d_h).$$

FlashAttention не превращает плотное внимание в линейное по длине. Материализующая реализация хранит $O(T^2)$ промежуточных элементов на голову, а FlashAttention требует $O(T)$ дополнительной памяти сверх входов и выхода для статистик softmax. Матрицы Q_i, K_i, V_i и O_i размера $O(Td_h)$ остаются.

Основной выигрыш определяется числом обменов между HBM и быстрой памятью, а не уменьшением числа пар токенов. В модели вычислений исходной статьи, если M_{SRAM} — вместимость быстрой памяти

в скалярных элементах, стандартная реализация требует

$$\Theta(Td_h + T^2)$$

обращений к HBM, а FlashAttention —

$$\Theta\left(\frac{T^2 d_h^2}{M_{\text{SRAM}}}\right)$$

при $d_h \leq M_{\text{SRAM}} \leq Td_h$ [4].

Метод особенно важен при обучении и обработке длинного входа, когда одновременно вычисляется много строк запросов. При генерации одного токена матрица запросов имеет одну строку на последовательность, а время часто определяется чтением весов и KV-кеша. Поэтому одинаковое ускорение во всех режимах не гарантируется. FlashAttention не уменьшает сам KV-кеш: кеш и блочное вычисление внимания решают разные задачи.

7. Сравнение вычислительных затрат

Метод	Экономия	Что остаётся	Основное ограничение
Полное дообучение	Нет ограничения на обновление	Все веса, градиенты, состояния оптимизатора, активации	Высокая память обучения и отдельный полный набор параметров
LoRA	Градиенты и состояния только для адаптеров	Базовые веса, основное умножение, активации	Ранг и выбранные проекции ограничивают поправку
QLoRA	Базовые веса хранятся в 4 битах	Вычисления после деквантизации, адаптеры, активации	Ошибка квантования и служебные данные блоков
KV-кеш	Нет повторного вычисления K, V префикса	Линейный рост памяти по длине и внимание ко всему контексту	Память кеша при длинном контексте и большом батче
FlashAttention	Нет полной матрицы внимания в HBM; меньше обменов	Плотное внимание с $O(T^2 d_h)$ операций	Выигрыш зависит от формы матриц и оборудования

Таблица 3. Разные методы сокращают разные компоненты стоимости. Их эффекты могут сочетаться, но не заменяют друг друга

Для одного проекта полезно разделять по крайней мере четыре бюджета.

1. Память и время обучения.
2. Размер сохраняемой части для конкретной задачи.
3. Память весов и KV-кеша при инференсе.
4. Задержка и пропускная способность отдельно для обработки входа и генерации.

Выбор метода начинается с ограничивающего бюджета. Например, LoRA уменьшает состояния оптимизатора, но не решает проблему квадратичной промежуточной памяти обычной реализации внимания. FlashAttention уменьшает эту промежуточную память, но не сокращает число параметров для конкретной задачи.

8. Оценка качества и эффективности

Эксперимент по дообучению должен одновременно измерять качество и ресурсы. Для сравнения полного дообучения, LoRA и QLoRA фиксируются:

- одна базовая модель и один токенизатор;
- одинаковые обучающие, валидационные и тестовые данные;
- функция потерь и правило маскирования токенов;
- число шагов, эффективный размер батча и порядок настройки гиперпараметров;
- метрика прикладной задачи или вероятность на отложенных ответах;
- число обучаемых параметров, размер сохранённого адаптера, пиковая память и время шага.

Ранг и набор адаптируемых проекций выбираются по валидационным данным. Тестовая выборка не используется для выбора конфигурации. Для генеративной задачи одной функции потерь недостаточно: дополнительно применяется метрика задачи и, если требуется, оценка человеком по заранее заданным критериям.

При сравнении инференса фиксируются оборудование, версия ядра, числовой формат, батч, T_x, T_y и метод выбора токена. Отдельно измеряются:

- время до первого токена;
- среднее время одного следующего токена;
- число обработанных и сгенерированных токенов в секунду;
- пиковая память, включая веса и KV-кеш;
- совпадение выхода с эталонной реализацией в заданном допуске.

FlashAttention является точным алгоритмом в вещественной арифметике. В конечной точности небольшие различия возможны из-за другого порядка операций. Их следует отделять от изменений качества, вызванных квантизацией весов.

9. Практические ограничения

9.1. ОГРАНИЧЕНИЕ РАНГА

LoRA предполагает, что полезную поправку к выбранной матрице можно хорошо описать малым рангом. Это предположение проверяется результатом на отложенных данных. Увеличение ранга и добавление проекций расширяют адаптер, но также увеличивают память и стоимость обучения. Универсального ранга или набора матриц для всех моделей и задач нет.

9.2. КВАНТИЗАЦИЯ БАЗОВОЙ МОДЕЛИ

QLoRA хранит приближение исходных весов. Ошибка зависит от распределения значений по блокам, размера блока и кодовой таблицы. Сильные выбросы и отличие распределения весов от предположений NF4 могут увеличить ошибку. При оценке необходимо сравнивать не только QLoRA с полным дообучением, но и квантизованную базовую модель с исходной.

9.3. АКТИВАЦИИ И ДЛИНА ПОСЛЕДОВАТЕЛЬНОСТИ

Малое число обучаемых параметров не устраняет промежуточные активации. Их память растёт с батчем, длиной последовательности и числом слоёв. Повторное вычисление активаций уменьшает число сохраняемых промежуточных значений. FlashAttention отдельно сокращает промежуточные данные внимания. Эти эффекты складываются, но относятся к разным частям памяти.

9.4. РЕЖИМ ИНФЕРЕНСА

Обработка входа и генерация имеют разные формы матричных операций. Оптимизация, полезная для большого блока запросов, может дать меньший эффект при одной новой позиции. Аналогично слияние LoRA удобно для одного адаптера, но усложняет обслуживание множества адаптеров в одном батче. Вывод о скорости делается только по измерению в целевом режиме.

10. Основные выводы

1. LoRA сохраняет функцию потерь и базовые веса, а обновление каждой выбранной матрицы задаёт произведением двух обучаемых факторов.
2. Число параметров одного LoRA-адаптера равно $r(d_{\text{in}} + d_{\text{out}})$, но базовые веса, основное умножение и память активаций остаются.
3. QLoRA хранит замороженную базовую модель в NF4, деквантизует блоки в вычислительный формат и обучает только параметры LoRA.

4. Обработка входа выполняется по позициям параллельно, а токены ответа генерируются последовательно. KV-кеш устраняет повторное вычисление префикса, но растёт линейно по его длине.
5. FlashAttention вычисляет точное плотное внимание блоками и не материализует полную матрицу вероятностей в HBM.
6. FlashAttention уменьшает дополнительную память и обмен данными, но сохраняет $O(T^2 d_h)$ арифметических операций для плотного внимания.
7. Качество, пиковая память, размер адаптера и скорость измеряются отдельно. Доля обучаемых параметров не заменяет ресурсный эксперимент.

A. Пример расчёта памяти обучения

Рассмотрим упрощённую модель с $P = 7 \times 10^9$ параметров. Пусть веса и градиенты хранятся по 2 байта, а два состояния Adam — по 4 байта. Без учёта активаций и дополнительной полной копии весов оценка полного дообучения равна

$$M_{\text{full}} = P(2 + 2 + 4 + 4) = 84 \times 10^9 \text{ байт}.$$

Если оптимизатор хранит ещё одну FP32-копию весов, добавляется 28×10^9 байт. Это показывает, почему одного размера файла параметров недостаточно для оценки памяти обучения.

Пусть LoRA содержит $P_{\text{L}} = 20 \times 10^6$ параметров. При тех же форматах параметры, градиенты и два состояния адаптера требуют

$$P_{\text{L}}(2 + 2 + 4 + 4) = 240 \times 10^6 \text{ байт}.$$

Дополнительно хранятся базовые веса: в BF16 это примерно 14×10^9 байт. К сумме добавляются активации и рабочая память.

Для QLoRA идеальные 4 бита на параметр дали бы

$$P\frac{4}{8} = 3{,}5 \times 10^9 \text{ байт}$$

для базовых весов. Реальное значение больше из-за масштабов блоков, метаданных и рабочих буферов. Адаптеры и активации также остаются. Числа в примере являются разложением компонентов, а не гарантией того, что модель поместится в ускоритель заданного объёма.

B. Проверка потокового softmax

Обозначим множество индексов, обработанных после b блоков, через

$$S_t^{(b)} = J_t^{(1)} \cup \cdots \cup J_t^{(b)}.$$

Докажем по индукции инварианты

$$m_{t,i}^{(b)} = \max_{j \in S_t^{(b)}} r_{t,j,i},$$

$$\ell_{t,i}^{(b)} = \sum_{j \in S_t^{(b)}} e^{r_{t,j,i} - m_{t,i}^{(b)}},$$

$$\tilde{o}_{t,i}^{(b)} = \sum_{j \in S_t^{(b)}} e^{r_{t,j,i} - m_{t,i}^{(b)}} v_{j,i}.$$

Для $b = 1$ равенства следуют из определения максимума, суммы и числителя на первом блок. Предположим, что они верны после $b - 1$ блоков. Новый максимум равен максимуму прежнего значения и логитов нового блока. Для любого ранее учтённого индекса

$$e^{r_{t,j,i} - m_{t,i}^{(b)}} = e^{m_{t,i}^{(b-1)} - m_{t,i}^{(b)}} e^{r_{t,j,i} - m_{t,i}^{(b-1)}}.$$

Поэтому прежние сумма и числитель пересчитываются умножением на $e^{m_{t,i}^{(b-1)} - m_{t,i}^{(b)}}$. После добавления вкладов нового блока получаются требуемые инварианты.

После последнего блока $S_t^{(R_t)} = \{j : j \le t\}$. Следовательно,

$$\frac{\tilde{o}_{t,i}^{(R_t)}}{\ell_{t,i}^{(R_t)}} = \frac{\sum_{j \le t} e^{r_{t,j,i} - m_{t,i}^{(R_t)}} v_{j,i}}{\sum_{j \le t} e^{r_{t,j,i} - m_{t,i}^{(R_t)}}} = \sum_{j \le t} \frac{e^{r_{t,j,i}}}{\sum_{u \le t} e^{r_{t,u,i}}} v_{j,i}.$$

Общий множитель $e^{-m_{t,i}^{(R_t)}}$ сокращается. Получен выход обычного причинного внимания.

Список литературы

1. LoRA: Low-Rank Adaptation of Large Language Models / E. J. Hu [и др.] // International Conference on Learning Representations (ICLR). — 2022. — arXiv: 2106.09685 [cs.CL]. — URL: https://arxiv.org/abs/2106.09685.
2. QLoRA: Efficient Finetuning of Quantized LLMs / T. Dettmers [и др.] // Advances in Neural Information Processing Systems (NeurIPS). — 2023. — arXiv: 2305.14314 [cs.LG]. — URL: https://arxiv.org/abs/2305.14314.
3. Efficiently Scaling Transformer Inference / R. Pope [и др.] // Proceedings of Machine Learning and Systems (MLSys). — 2023. — arXiv: 2211.05102 [cs.LG]. — URL: https://arxiv.org/abs/2211.05102.
4. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness / T. Dao [и др.] // Advances in Neural Information Processing Systems (NeurIPS). — 2022. — arXiv: 2205.14135 [cs.LG]. — URL: https://arxiv.org/abs/2205.14135.